Binaml: Continual Learning over Binary Feature Streams

Romain Zimmer
hi@romainzimmer.com

2026-08-11 (Draft)

Abstract

Binaml focuses on continual learning models over streams of binary features subject to data drift. Binary features provide a task-agnostic representation and enable models optimized for memory, latency, and energy efficiency. This paper specifies Binaml's online ensemble models, `BRegressor` for scalar regression and `BClassifier` for multiclass classification, as ensembles of batch-learned boolean experts with task-specific linear heads. Both models adapt online rather than relying on a stationary train/serve assumption and learn compact compositional representations from residual feedback. Runtime memory is fixed at model construction and the online hot path performs zero heap allocation after `new`. The models are environment-agnostic: they do not depend on how the stream is produced. This paper specifies the current models and reports reproducible prequential comparisons on versioned synthetic regression and classification streams.

1 Notation

Symbols below are used consistently in this paper and in `crates/binaml-core`.

d, x_t Input width and binary input (`source_feature_count` / `n_features`).

$y_t, \hat{y}_t$ Scalar regression target and prediction.

$C, c_t, \hat{c}_t$ Number of classes, class label, and predicted class (`n_classes`).

F, F_t Current expert count and expert count after batch finalizations through time t (`function_count()`); $F \leq K_{\max}$.

e_t, s_t Residual and supervision label appended to the expert batch.

b, w_k, η, λ Intercept, ensemble weight of expert k, learning rate (`learning_rate`), and ℓ_2 decay (`l2`).

$f_k(x)$ Boolean expert k mapping $\{0,1\}^d$ to $\{0,1\}$, stored as a compact `FunctionGraph`.

$N_k, N_{\max}$ Compact node count in expert k and fixed per-expert node cap (`max_expert_nodes`).

$T \in \{0, \ldots, 15\}$ Four-bit truth table of an arity-2 composed node.

B, S, n Expert batch size (`batch_size`), SGD steps per observation (`sgd_steps`), and batch index.

$K_p, K_{\max}, l_{\text{pat}}$ Parent top-K_p width (`parent_top_k`), ensemble capacity (`max_functions`), and layer patience (`l_pat`): stop after l_{pat} consecutive composed layers without global in-sample accuracy improvement.

P, V, L_{build} Derived batch-build caps at construction: pair-scratch width $P = \binom{K_p}{2}$; ephemeral graph pool $V = (K_p \wedge d) + L_{\text{build}} K_p + 1$ with $L_{\text{build}} = \max(1, N_{\max} - K_p \wedge d)$.

2 Model

Both `BRegressor` and `BClassifier` maintain an ensemble of boolean experts. Expert structure evolves slowly through batch-local graph growth, in-sample output selection, and compaction into `FunctionGraph` objects; expert weights in the linear head adapt on every observation. At capacity $K_{\max}$, ensemble pruning drops the weakest expert. The two models differ only in the linear head that maps expert activations to predictions.

2.1 Regression head

At time t, the regressor receives $x_t \in \{0,1\}^d$ and later observes a finite scalar y_t. If useful signal is sparse weighted boolean structure over bits, a linear head over learned boolean indicators is a direct hypothesis class: continuous amplitudes, discrete experts, and an inspectable composition. Let F_t be the number of experts after batch finalizations through time t. The prediction is

$$\hat{y}_t = b + \sum_{k=1}^{F_t} w_k \, f_k(x_t),$$

where $b \in \mathbb{R}$, each $w_k \in \mathbb{R}$, and each $f_k(x_t) \in \{0,1\}$. Each f_k is a compact boolean expert built from one supervision batch. Expert construction consumes a binary supervision bit derived from the scalar target. With pre-update prediction $\hat{y}_t$, the residual is $e_t = y_t - \hat{y}_t$ and the label appended to the current expert batch is

$$s_t = \mathbf{1}\{e_t \geq 0\}.$$

This bit is computed with the current head parameters before the linear updates on step t.

Weight updates. For one SGD step with learning rate $\eta > 0$ and decay $\lambda \geq 0$, first apply ℓ_2 decay to every stored weight, then update the intercept and weights from the pre-step residual:

$$\begin{aligned} w_k &\leftarrow (1 - \eta\lambda)\, w_k && \text{for all stored } k, \\ b &\leftarrow b + \eta e_t, \\ w_k &\leftarrow w_k + \eta e_t f_k(x_t). \end{aligned}$$

Each observation performs S such steps on the same activation row.

2.2 Classification head

The classifier receives $x_t \in \{0,1\}^d$ and later observes a class label $y_t \in \{0, \ldots, C-1\}$. Let F_t be the number of experts after batch finalizations through time t. For each class c, logits are

$$\ell_{t,c} = b_c + \sum_{k=1}^{F_t} w_{k,c} \, f_k(x_t),$$

where $b_c \in \mathbb{R}$ and each $w_{k,c} \in \mathbb{R}$. The predicted class is $\hat{c}_t = \operatorname{argmax}_c \ell_{t,c}$, with ties broken toward the smallest index. Expert construction consumes a binary supervision bit derived from the class label.

With pre-update prediction $\hat{c}_t$, the label appended to the current expert batch is the misclassification indicator

$$s_t = \mathbf{1}\{\hat{c}_t \neq y_t\}.$$

This bit is computed with the current head parameters before the linear updates on step t.

Weight updates. Let $p_{t,c} = \mathrm{softmax}(\ell_t)_c$ be the pre-step class probabilities. For one SGD step on softmax cross-entropy with learning rate $\eta > 0$ and decay $\lambda \geq 0$, define the class errors $\delta_{t,c} = p_{t,c} - \mathbf{1}\{c = y_t\}$. First apply ℓ_2 decay to every stored weight, then update intercepts and weights:

$$\begin{aligned} w_{k,c} &\leftarrow (1 - \eta\lambda)\, w_{k,c} && \text{for all stored } k, c, \\ b_c &\leftarrow b_c - \eta\, \delta_{t,c}, \\ w_{k,c} &\leftarrow w_{k,c} - \eta\, \delta_{t,c}\, f_k(x_t). \end{aligned}$$

Each observation performs S such steps on the same activation row.

3 Building experts

Binaml builds and maintains a collection of boolean experts. Each expert is a compact boolean function over input bits; the ensemble grows one expert per filled batch while weights w_k update on every observation. Expert structure therefore evolves slowly, on batch boundaries, while the linear head tracks the task quickly. Composition starts from the most basic nontrivial boolean learning unit: two-input tables. Combining them yields richer experts without beginning from high-arity search. A composed node combines two parent activations using a four-bit truth table $T \in \{0, \ldots, 15\}$.

3.1 Expert representation and batch learning

Each expert batch grows an ephemeral narrow DAG: layer zero holds up to K_p non-constant source bits; at each subsequent layer the builder forms every distinct pair among the current layer's parents, learns a truth table on the current batch, and keeps the top-K_p candidates by descending |assoc|. Growth stops after L_{build} composed layers, after l_{pat} consecutive composed layers without global in-sample accuracy improvement, when fewer than two parents remain, when no valid pair survives constant filtering, or once accuracy is perfect on $\mathcal{B}_n$. When the batch fills, the builder selects one output node from the top-K_p nodes by |assoc|, then by descending in-sample accuracy, inverting the output bit when the inverted node is more accurate on $\mathcal{B}_n$. The subgraph is compacted into a `FunctionGraph` and appended to the ensemble with weight zero. If the ensemble exceeds capacity $K_{\max}$, the expert with smallest $|w_k|$ is removed, breaking ties toward the oldest index. There is no hold-out batch and no shared candidate queue across batches.

3.2 Compact evaluation

A stored `FunctionGraph` lists source indices and composed nodes in topological order. Each node is either a source coordinate, a constant, or a composed operation referencing two already evaluated positions and one four-bit truth table. One forward pass evaluates $f_k(x)$ for prediction and SGD. The eight `u8` counters used to learn a table cover every $(u, v, s) \in \{0, 1\}^3$ combination, which limits the expert batch size to $B \leq 255$.

3.3 Supervision and batching

Exact truth-table counting needs a bounded window; batching amortizes graph growth. Batch n is

$$\mathcal{B}_n = \{(x_t, s_t)\}_{t \in I_n}, \qquad |I_n| = B,$$

where I_n is the contiguous block of B observations since the previous structural update. Processing $\mathcal{B}_n$ grows an ephemeral graph, selects one output by in-sample accuracy, compacts it, appends $(f, w = 0)$ to the ensemble, and prunes to $K_{\max}$ if needed.

Association score. For a boolean batch column $z_{1:B}$ and supervision labels $s_{1:B}$, let $N = B$, $N_x = \sum_i z_i$, $N_y = \sum_i s_i$, and $N_{xy} = \sum_i z_i s_i$. Define

$$\mathrm{assoc}(z, s) = N\, N_{xy} - N_x N_y.$$

Parent nodes are ranked by $|\mathrm{assoc}(z, s)|$, breaking ties by node id. Sources use their coordinate columns; composed nodes use their evaluated batch columns. Negation at layer zero is not materialized as separate nodes: anti-correlated sources rank highly on |assoc|, and output polarity flip handles $\neg x_i$ when it improves in-sample accuracy.

3.4 Truth-table learning

Given two parent batch columns $u_{1:B}$, $v_{1:B}$ and supervision labels $s_{1:B}$, the learner returns the Hamming-error-minimizing four-bit table. Its state is a fixed array of eight `u8` counters, one for each combination of the two parent values and supervision label. The learned result is a four-bit truth table represented by one `u8`. The counters record the frequencies needed to select each table output by majority, with ties resolving to zero. As one counter may receive every observation, the `u8` counters bound the admissible batch size to $B \leq 255$.

3.5 Batch graph growth

Starting from layer zero, the builder repeatedly appends composed layers until L_{build} layers exist or a structural stop triggers:

1. Let $L_{\text{build}} = \max(1, N_{\max} - (K_p \wedge d))$.
2. While the composed-layer count is below L_{build}, fewer than l_{pat} consecutive layers have failed to improve global in-sample accuracy, at least two non-constant parents remain, at least one valid pair survives constant filtering, and accuracy is not yet perfect on $\mathcal{B}_n$:
   (a) Let $P_{\ell-1}$ be the nodes in layer $\ell - 1$ (already capped at K_p).
   (b) Form every distinct pair in $P_{\ell-1}$.
   (c) For each pair, learn a truth table on $\mathcal{B}_n$, discard constant tables and constant parents, and rank survivors by |assoc|.
   (d) Keep the top-K_p composed nodes as layer ℓ.

After growth stops, the builder selects the output node from the top-K_p surviving nodes by |assoc|, then by descending in-sample accuracy, inverting the output when that improves accuracy on $\mathcal{B}_n$.

3.6 Ensemble pruning

When appending a new expert would exceed $K_{\max}$, remove the stored index with smallest $|w_k|$, breaking ties toward the oldest index. Zero-weight experts can therefore rotate at capacity even when still structurally useful on past batches.

4 Runtime data structures and memory

The online models preallocate all hot-path storage at construction from $(d, B, K_{\max}, N_{\max}, K_p, l_{\text{pat}}, C)$; derived caps (P, L_{build}, V) follow § Notation. After `new`, the ensemble `predict`/`update` loop and in-place expert batch build perform **zero heap allocation**: no `Vec` growth, hash maps, or per-call scratch vectors on those paths. Standalone batch learning exposes the same builder through `FunctionBuildSession`, which owns one `BuildWorkspace`; its `build` method is zero-alloc after session construction. The one-shot `FunctionBuilder::build` entry point still materializes a heap `FunctionModel` for tests and scripting.

Persistent model state. `BRegressor` and `BClassifier` wrap a `BooleanEnsemble` with a task-specific head and a fixed `EnsembleWorkspace`. Experts live in `functions: Box<[FunctionGraph]>` with capacity $K_{\max}$; active count $F \le K_{\max}$ is tracked in the head. Pruning copies slot $j{+}1$ into slot j and clears the trailing slot in place. Each `FunctionGraph` stores `source_indices: Box<[usize]>` (capacity d), `nodes: Box<[CompactNode]>` (capacity $N_{\max}$), live counts `n_sources`/`n_nodes`, output index, and optional output inversion. `CompactNode` is a source coordinate, a constant bit, or a composed gate `{first, second, truth_table}` referencing prior positions.

Type	Fields	Capacity	Role
`EnsembleConfig`	η, λ, S, B, K_p, $K_{\max}$, $N_{\max}$, l_{pat}	fixed	hyperparameters
`RegressionHead`	`intercept`, `weights: Box<[f64]>`, `active`	$K_{\max}$ weights	linear head
`ClassificationHead`	C, `intercepts`, `weights: Box<[f64]>`, `active`	$C{+}K_{\max}C$ floats	softmax head
`FunctionGraph`	sources, nodes, counts, output, invert	d, $N_{\max}$	one expert f_k
`BooleanEnsemble`	config, head, `functions`, `workspace`	$K_{\max}$ slots	online engine

Ensemble workspace (hot path). `EnsembleWorkspace` is allocated once in `BooleanEnsemble::new` and reused across observations:

Buffer	Shape	Use
`function_values`	$K_{\max}$ bool	latest $f_k(x_t)$
`pending_features`	d bool	`predict` pairing
`pending_function_values`	$K_{\max}$ bool	`update` SGD row
`eval_scratch`	$N_{\max}$ bool	expert forward pass
`logits`, `probabilities`	C f64 each	classifier head
`batch_features`	Bd bool	column-major partial batch
`batch_signs`	B bool	supervision bits
`build`	`BuildWorkspace`	in-place batch graph build

Batch columns are exposed to the builder as a flat `SignBatch` view over `batch_features` without allocating column pointer vectors.

Batch-build workspace. Each `BuildWorkspace` holds fixed pools sized by $(V, P, K_p, L_{\text{build}}, N_{\max}, d)$: `nodes` ($V$ `EphemeralNode`), `association_scores`/`accuracy_scores` (V), flat layer storage `layer_ids` ($K_p(L_{\text{build}}{+}1)$) with `layer_ends`, `parent_buf` and `rank_scratch` (K_p and V), `FlatColumnCache` ($2K_p$ column slots $\times$ B), pair scratch `pair_scratch` (P `FeatureCounter`) and `pair_candidates` (P), and compaction buffers `compact_nodes` ($N_{\max}$), `compact_sources` (d), plus reachability/order

scratch (V each). `build_in_workspace` resets lengths and reuses these buffers; compaction writes directly into a preallocated expert slot via `reset_from_parts`. The legacy heap `EphemeralGraph` type remains for one-shot `FunctionBuilder::build` and tests only.

Type	Fields	Size	Lifetime
`FunctionBuildSession`	`BuildWorkspace`, config	$O(V+PK_pB)$	reusable builder
`SignBatch`	columns or flat Bd layout	borrowed	one build
`FeatureCounter`	`counts: [u8; 8]`	8 B	one gate learn
`FunctionModel`	`EphemeralGraph`, output	heap	one-shot API only

Memory scaling and allocation policy. Ignoring slice headers, resident ensemble memory scales as $8K_{\max}+FC$ bytes for regression weights or $8C(1+K_{\max})$ for classification, plus $K_{\max}(8d+24N_{\max})$ for expert slots (using $N_{\max}$ as the stored node cap), plus hot/build workspace $O(K_{\max}+d+N_{\max}+C+Bd+PK_pB+V)$ bytes derived at construction. Structural work peaks once every B updates inside the preallocated `build` pool; per-observation work reuses `eval_scratch` and head scratch without heap growth. Regression tests assert zero allocations after `new` on the ensemble and `FunctionBuildSession` hot paths.

5 Online head updates

Expert identity is discrete; head parameters are continuous, so the two are updated separately. For each observation, `predict` evaluates the current experts once; `update` then computes the head-specific supervision bit s_t, appends (x_t, s_t) to the current expert batch, and performs S single-sample gradient steps on that activation row using the task-specific rules above. In both models the newly appended expert at a batch boundary is excluded from SGD on the batch-fill step because it is added only after those updates. There is no gradient through boolean structure: SGD tracks task loss at the sample level, while structure search runs when enough supervision bits have accumulated. After every B new observations, the batch is consumed to grow, select, compact, and append one expert, then cleared.

6 Online learning procedure

Configuration

Linear Learning rate η, decay λ, and S SGD steps per observation.

Batch structure Expert batch size B, parent top-K_p width K_p, derived caps (P, L_{build}, V), patience l_{pat}, node cap $N_{\max}$, and ensemble capacity $K_{\max}$.

Initialize: empty ensemble, $b \leftarrow 0$, and empty expert-batch buffers.
For each observation (x_t, y_t):

1. Validate the input width and task target.
2. Evaluate each expert $f_k(x_t)$ and form the task prediction ($\hat{y}_t$ or $\hat{c}_t$).
3. Compute the head-specific supervision bit s_t and append (x_t, s_t) to the current expert batch.
4. Apply S single-sample head updates using the current activation row.
5. After B accumulated observations, grow a graph on the batch, select and compact one output, append it with weight zero, prune to $K_{\max}$ if needed, and clear the expert batch.

Cost. Prediction and activation evaluation are linear in the number of experts and their node counts. Each linear step costs one expert evaluation. Structural work occurs once per expert batch: graph growth scans selected parent pairs over B rows, then compaction produces one expert.

7 API and integration boundaries

The `binaml.models` package owns the online models and has no environment dependency: the goal is generic models, decoupled from where the data comes from and how it was produced. The public surfaces are `BRegressor` and `BClassifier`, which implement `predict(features)` and `update(target)` for binary feature vectors of width d. The prediction call retains pairing state; `update` must follow a preceding `predict` and applies the revealed target to that stored step. Expert construction is internal to `update` once a batch fills. Evaluation packages compose models with environments through predict-then-update prequential evaluation. Named benchmarks only compose these independent components.

8 Prequential evaluation protocol

For each emitted sample, an evaluator first records the model prediction, then exposes the target through `update`. Regression benchmarks compute mean squared error from the recorded pre-update predictions; classification benchmarks compute accuracy from the recorded pre-update class predictions. Structure updates occur only after a full expert batch fills inside `update`; residual signs are measured with the current head parameters before the corresponding updates. Hyperparameters, implementation version, and the online protocol are required to interpret any reported trajectory.

9 Evaluation

The benchmark inputs and model configurations are versioned in this paper's benchmark directory.

9.1 Regression

The regression task uses 1,000 observations and seeds $\{0, 1, 2, 3, 4\}$ with 32 input bits, 12 fixed target components, noise standard deviation 0.1, and the same function, DAG, and target-scale settings. Input, gate, and intercept distributions each resample independently with probability 0.01.

All models receive the scenario width. `BRegressor` uses learning rate $\eta = 3 \times 10^{-2}$, ℓ_2 decay 5×10^{-4}, expert batch size $B = 16$, twenty single-sample SGD steps, parent top-$K_p = 8$, and ensemble capacity $K_{\max} = 64$. Each expert enters the linear head as $f_k(x) \in \{0, 1\}$. The linear baseline is a JAX replay-batch implementation using learning rate 0.03, the same decay, replay size 32, and step count, with uncentered binary inputs. The MLP baseline is a JAX one-hidden-layer ReLU network using replay size 32 and three full-batch SGD steps; one 50-unit hidden layer; constant learning rate 0.003, $\alpha = 10^{-4}$, and random state zero. Linear and MLP baselines share the same float32 replay SGD scaffold.

Tables report the mean ± standard error over five seeds. MSE is the optimized loss; RMSE expresses the same error in target units. Total time is the prequential predict-plus-update time for a 1,000-sample trajectory.

Model	MSE	RMSE	total s
BRegressor (ours)	$\mathbf{0.400 \pm 0.011}$	$\mathbf{0.632 \pm 0.009}$	$\mathbf{0.021 \pm 0.000}$
Linear SGD (JAX)	0.825 ± 0.024	0.908 ± 0.013	4.769 ± 0.015
MLP SGD (JAX)	0.907 ± 0.018	0.952 ± 0.009	6.373 ± 0.157

9.2 Classification

The classification task uses the same input width, component count, noise standard deviation, drift probabilities, and function settings as the regression default, with three classes and per-class gates, weights, and intercepts. Scenario and model entries are versioned in `benchmarks/scenarios/default_multiclass.` and `benchmarks/models_classification.json`. `BClassifier` uses learning rate $\eta = 1.2 \times 10^{-1}$, ℓ_2 decay 5×10^{-4}, expert batch size $B = 12$, thirty single-sample SGD steps, and the same structural limits as `BRegressor`; linear and MLP classifier baselines mirror their regression counterparts with softmax cross-entropy. Reported comparisons use prequential accuracy over the same seed set after optional warm-up exclusion.

Tables report the mean ± standard error over five seeds. Accuracy is the fraction of correct pre-update class predictions on the 900 post-warm-up observations. Total time is the prequential predict-plus-update time for a 1,000-sample trajectory.

Model	accuracy	total s
BClassifier (ours)	$\mathbf{0.765 \pm 0.021}$	$\mathbf{0.035 \pm 0.001}$
Linear SGD (JAX)	0.653 ± 0.038	0.811 ± 0.012
MLP SGD (JAX)	0.645 ± 0.041	1.114 ± 0.007

10 Limitations and scope

This document specifies the ensemble models used by Binaml. It does not claim optimality of residual-sign learning or in-sample expert selection. Ensemble churn at capacity can drop zero-weight experts that remain useful on past batches; in-sample output selection trades hold-out promotion for simpler batch boundaries. The reported regression comparison is limited to the versioned synthetic task, fixed configurations, and reproducible baseline runs described above.